

Лабораторна робота № 5

Вивчення UART мікроконтролерів PSoC 6 фірми Cypress

Мета роботи: Вивчити функціонування USB UART з'єднання для забезпечення передачі даних між PC та мікроконтролером PSoC 6, реалізація проектів з використанням компоненти UART.

Теоретичні відомості

Розглянемо проект, в якому використовується процесорне ядро Cortex-M4 (CM4) мікроконтролера PSoC 6 для передачі даних по UART та управління світлодіодами. При перезавантаженні пристрою процесор Cortex-M0+ (CM0+) включає процесор CM4. ЦП CM4 використовує компоненту UART для передачі повідомлення на термінал комп'ютера та при натисканні клавіші "Enter", світлодіод на стенді починає блимати.

На рис. 5.1 зображена схема проекту в середовищі PSoC Creator 4.2 для цього проекту.

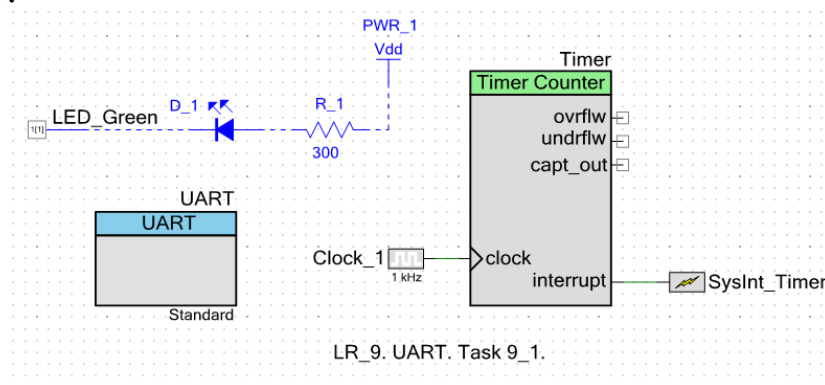


Рис. 5.1. Схема проекту з компонентою UART

Компонента TCPWM_Counter_PDL таймера/лічильника (TCPWM) налаштована як таймер і генерує переривання кожну секунду. Вхід синхронізації компоненти TCPWM під'єднаний до джерела тактових імпульсів частотою 1 кГц.

Вона дозволяє швидко налаштовувати обладнання TCPWM для роботи з таймером/лічильником. Найчастіше використовується для вимірювання часових інтервалів або підрахунку зовнішніх подій.

Параметри компоненти TCPWM_Counter_PDL детально розглянуті в ЛР № 4. Вікно налаштування параметрів цієї компоненти зображено на рис. 5.2.

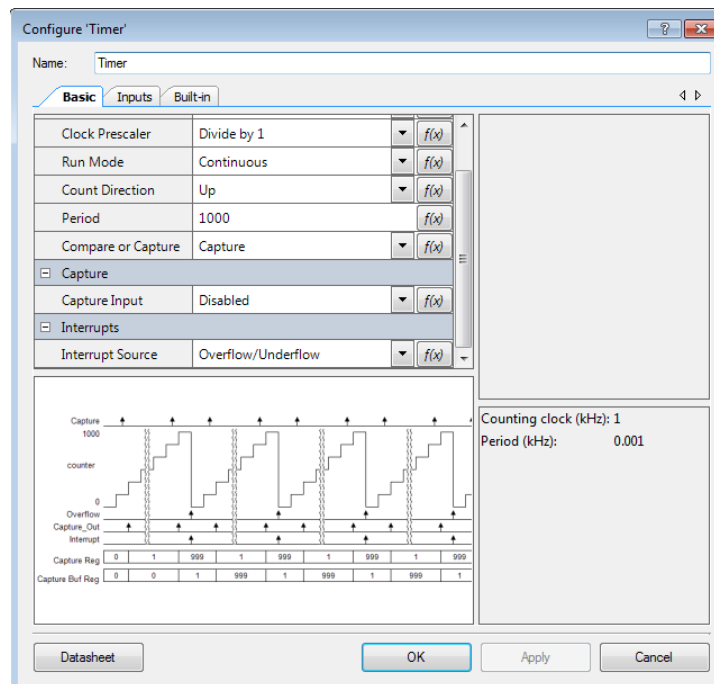


Рис. 5.2. Налаштування конфігурації компоненти таймера (TCPWM)

Компонента *SCB_UART_PDL* забезпечує асинхронний зв'язок, який отримав назву RS232 або RS485. Ця компонента може бути конфігурована для двохсторонньої передачі даних (Full Duplex), тільки для отримання мікроконтролером PSoC 6 даних (RX) або тільки для передачі з мікроконтролера PSoC 6 даних (TX). Її також можна налаштувати як інтерфейс SmartCard або IrDA.

Для більшості випадків використання можна легко налаштувати UART, вибравши швидкість передачі даних, парність, кількість біт даних і кількість старт бітів. Найпоширеніша конфігурація для RS232 часто вказується як "8N1", що скорочено означає вісім біт даних, без паритету та одного стоп біта. Це конфігурація є за замовчуванням для компоненти *SCB_UART_PDL*. Тому в більшості програм потрібно лише встановити швидкість передачі даних. Іншим поширеним використанням UART є багатоабонентські мережі RS485. Компонента *SCB_UART_PDL* підтримує 9-бітовий режим адресації з апаратним визначенням адреси.

Інтерфейси UART існували вже давно. Тому з часом було багато їх варіацій на фізичному рівні та рівні протоколу. Сюди входять, але не обмежуються ними, RS423, DMX512, MIDI, шина LIN, застарілі термінальні протоколи. Для підтримки часто використовуваних варіантів UART компонента забезпечує підтримку конфігурації для кількості біт даних, стоп бітів, парності, апаратного управління потоком, а також генерації та виявлення парності.

Компонента *SCB_UART_PDL* - це об'єкт графічної конфігурації, побудована на основі драйвера *su_scb*, доступного в бібліотеці периферійних драйверів (PDL). Це дозволяє виконувати підключення на основі схеми і конфігурацію пристрою, як визначено в діалоговому вікні Component Configure.

Компонента SCB_UART_PDL використовується тоді, коли потрібен сумісний інтерфейс асинхронного зв'язку, особливо RS232 і RS485 та інші варіанти. Також можна використовувати компоненту для створення більш просунутих асинхронних протоколів, таких як Smartcard, IrDA, а також власних клієнтських або промислових протоколів.

Компоненту SCB_UART_PDL не потрібно використовувати у тих випадках, коли конкретна компонента вже створена для адресації протоколу. Наприклад, якщо надається LIN або MIDI компонента, вона має конкретну реалізацію, що забезпечує функціональність як апаратного, так і протокольного рівня. В цьому випадку компонента SCB_UART_PDL не потрібна.

Визначення:

UART – (Universal Asynchronous Receiver Transmitter) універсальний асинхронний приймач-передавач, який зазвичай називають RS232. UART підтримує такі режими:

- UART - це стандартний режим.
- SmartCard - передача подібна до передачі UART, з додаванням підтвердження (NACK), яке може бути надіслано від приймача до передавача. NACK завжди дорівнює "0". І передавач, і приймач можуть управляти однією лінією введення/виведення, але тільки не одночасно.
- IrDA – протокол асоціації інфрачервоних даних. В ньому додається схема модуляції до сигналу UART. В передавачі біти модулюються. У приймачі біти демодулюються. Схема модуляції використовує формат з поверненням до нуля (RZI - Return-to-Zero-Inverted). Значення біта "0" сигналізується коротким імпульсом логічної "1" на лінії, а значення біта "1" сигналізується утримуванням лінії на рівні логічного "0".

Швидкий старт.

1. Перетягнути компоненту UART (SCB) з каталогу компонент Cypress/Communications/UART на схему. Екземпляр цієї компоненти отримує ім'я UART_1.

2. Двічі клацнути на екземплярі компоненти, щоб відкрити діалогове вікно Configure.

3. На вкладці Basic потрібно вибрати швидкість передачі даних, з якою буде виконуватися обмін даними через інтерфейс UART.

Для успішного зв'язку розрахована фактична швидкість передачі даних має бути в межах допустимої точності. Якщо це не так, потрібно повторити значення параметра Oversample, щоб досягти бажаної швидкості передачі даних.

4. В редакторі виводів (Pin Editor) потрібно вибрати виводи для інтерфейсу UART.

5. Для перевірки правильності дизайну апаратної частини потрібно виконати побудову (Build) проекту. Це додасть потрібні модулі PDL в Workspace Explorer та згенерує дані конфігурації для екземпляра UART_1.

6. В файлі main.c потрібно ініціалізувати периферійний пристрій та запустити додаток.

Параметр Interrupt вкладки Advanced компонента UART дозволяє вибирати між внутрішнім і зовнішнім розміщенням компоненти переривання.

Низькорівневий API.

Функції низького рівня дозволяють безпосередньо взаємодіяти з обладнанням і не використовують `Cy_SCB_UART_Interrupt`. Ці функції не потребують контексту для роботи. Це означає, що `NULL` можна передати для параметра контексту в `Cy_SCB_UART_Init` і `Cy_SCB_UART_Disable` замість вказівника на структуру контексту.

Для запису даних в TX FIFO можна використати одну з наступних функцій: `Cy_SCB_UART_Put`, `Cy_SCB_UART_PutArray`, `Cy_SCB_UART_PutArrayBlocking` або `Cy_SCB_UART_PutString`. Введення даних у TX FIFO починає передачу даних.

Для читання даних з RX FIFO використовують одну з наступних функцій:
 Cy_SCB_UART_Get, Cy_SCB_UART_GetArray або
 Cy_SCB_UART_GetArrayBlocking.

Стан прапорців можна опитувати за допомогою функцій:
Cy SCB UART GetRxFifoStatus і Cy SCB UART GetTxFifoStatus.

Також для опитування можна використовувати такі функції Cy_SCB_UART_IsTxComplete, Cy_SCB_UART_GetNumInRxFifo та Cy_SCB_UART_GetNumInTxFifo.

Розглянемо ці функції.

1. Cy_SCB_UART_Put(). Ця функція заносить один елемент даних в UART TX FIFO. Також вона повертає скільки елементів даних було записано в TX FIFO (0 або 1).

```
__STATIC_INLINE uint32_t Cy_SCB_UART_Put ( CySCB_Type * base,
                                           uint32_t      data
                                           )
```

Параметри:

base Вказівник на екземпляр UART SCB.

data Дані для запису в UART SCB. Розмір елемента визначається типом даних, який залежить від налаштованої ширини даних.

2. Cy_SCB_UART_PutArray(). Ця функція заносить масив даних у UART TX FIFO. Також вона повертає кількість елементів даних, розміщених у TX FIFO.

[illegible]

)

Параметри:

base Вказівник на екземпляр UART SCB.

buffer Вказівник на дані, які заносяться в TX FIFO. Розмір елемента визначається типом даних, який залежить від налаштованої ширини даних TX.

size Кількість елементів даних для передачі.

3. Cy_SCB_UART_Get(). Ця функція зчитує один елемент даних із UART RX FIFO. Вона не перевіряє, чи є дані в RX FIFO перед їх читанням. Якщо RX FIFO порожній, функція повертає CY_SCB_UART_RX_NO_DATA. Функція повертає дані з RX FIFO. Розмір елемента даних визначається налаштованою шириною даних.

```
__STATIC_INLINE uint32_t Cy_SCB_UART_Get(CySCB_Type const * base)
```

Параметри:

base Вказівник на екземпляр UART SCB.

4. Cy_SCB_UART_GetArray(). Ця функція зчитує масив даних із UART RX FIFO. Вона не блокується. Функція повертає, скільки елементів даних було прочитано з RX FIFO.

```
__STATIC_INLINE                                uint32_t  
Cy_SCB_UART_GetArray                          ( CySCB_Type const * base,  
                                                void *                buffer,  
                                                uint32_t            size  
                                                )
```

Параметри:

base Вказівник на екземпляр UART SCB.

buffer Вказівник на місце для розміщення даних, зчитаних з RX FIFO. Розмір елемента визначається типом даних, який залежить від налаштованої ширини даних TX.

size Кількість елементів даних для читання з RX FIFO.

5. Cy_SCB_UART_GetRxFifoStatus(). Ця функція повертає поточний статус RX FIFO.

```
__STATIC_INLINE uint32_t Cy_SCB_UART_GetRxFifoStatus ( CySCB_Type const * base )
```

Параметри:

base Вказівник на екземпляр UART SCB.

7. Виконати побудову проекту і програмування мікроконтролера PSoC 6.

Параметри компоненти.

Діалогове вікно налаштування компоненти SCB_UART_PDL дозволяє редагувати параметри конфігурації для екземпляра компоненти. Параметри розташовані на різних вкладках.

Основна вкладка.

Ця вкладка (рис. 5.3) містить параметри компоненти (табл. 5.1), які використовуються в загальних параметрах ініціалізації периферійного пристрою.

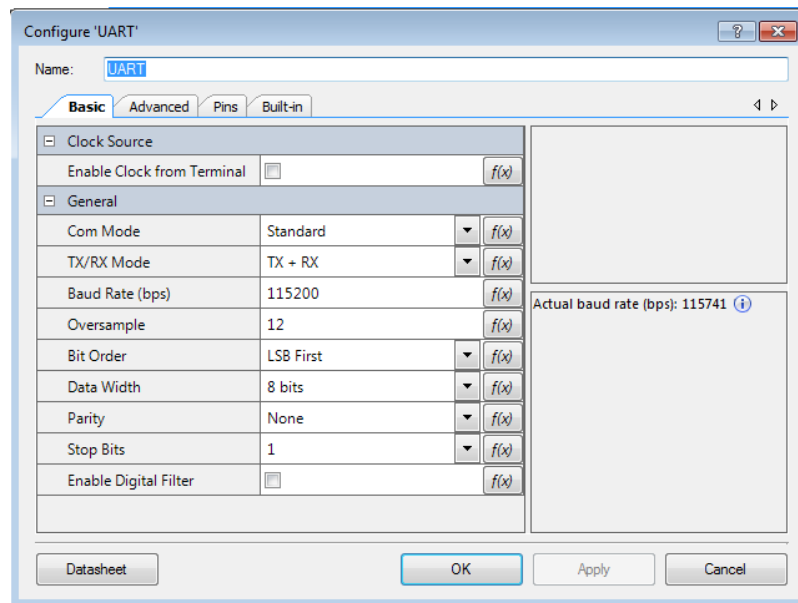


Рис. 5.3. Головна вкладка вікна налаштування параметрів компоненти SCB_UART_PDL

Табл. 5.1. Параметри головної вкладки вікна налаштування компоненти SCB_UART_PDL

Ім'я параметру	Опис
Enable Clock from Terminal	Цей параметр дозволяє вибирати між внутрішньо налаштованими тактовими імпульсами (компонентою) або зовнішніми тактовими імпульсами (користувачем) для роботи компоненти.
Com Mode	Цей параметр визначає підрежим UART: Standard, SmartCard або IrDA.
TX/RX Mode	Цей параметр включає функціональність приймача або передавача або обох одночасно.
Baud Rate (bps)	Цей параметр визначає швидкість передачі даних у біт/с. Фактична швидкість передачі даних може відрізнятися залежно від доступної тактової частоти та налаштувань компоненти. Діапазон: 1 - 1000000 біт/с.
Oversample	Цей параметр визначає, скільки тактових сигналів компоненти передискретизують вибрану швидкість передачі даних. Діапазон: 8 - 16 (крім режиму IrDA). Значення передискретизації попередньо визначені для режиму IrDA.
Actual baud rate	Фактична швидкість передачі даних відображає швидкість передачі даних, з якою компонента буде працювати з поточними налаштуваннями. Фактори, які впливають на розрахунок фактичної

	швидкості передачі даних: номінальна частота тактового сигналу компонента (внутрішнього чи зовнішнього) та параметр передискретизації.
Bit Order	Цей параметр визначає напрямок, в якому передаються послідовні дані. Якщо встановлено значення MSB, першим передається старший біт. Коли спочатку встановлено LSB, першим передається наймолодший біт.
Data Width	Цей параметр визначає ширину окремого елементу даних у бітах. Діапазон: 5-9.
Parity	Цей параметр визначає функціональність розташування біта парності під час передачі як None, Odd або Par.
Stop Bits	Цей параметр визначає кількість стоп бітів.
Enable Digital Filter	Цей параметр застосовує до вхідних ліній UART цифровий 3-го порядку медіанний фільтр.
Retry on Nack	Цей параметр визначає, чи надсилати повідомлення знову після отримання відповіді NACK. Використовується лише тоді, коли Com Mode встановлено в SmartCard.
Invert RX	Цей параметр дозволяє інверсію вхідного сигналу лінії RX. Використовується лише тоді, коли Com Mode встановлено в IrDA.
Low Power Receiving	Цей параметр включає опцію малопотужного приймача. Використовується лише тоді, коли Com Mode встановлено в IrDA.

Налаштування компоненти переривання SysInt для реалізації інтерфейсу переривання компоненти TCPWM з процесором CM4. Двічі клацнемо на ньому і в вікні, що відкриється, поміняємо ім'я на Isr_Timer (рис. 5.4).

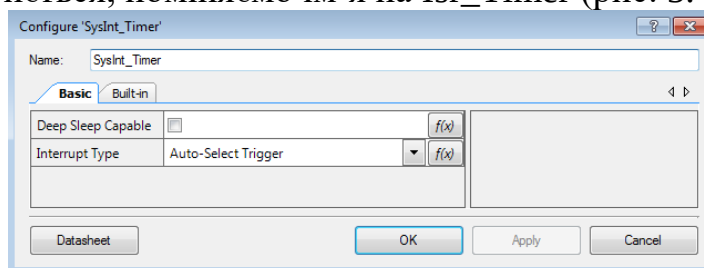


Рис. 5.4. Налаштування компоненти SysInt_PDL

Під'єднаємо вихід компоненти переривання SysInt_Timer до виходу переривання компоненти Timer. Це маршрутизує переривання TCPWM на ЦП CM4. Вибір ЦП CM4 для цього переривання встановимо в системі налаштування переривання пізніше.

Встановлення фізичних виводів для кожної компоненти. Вибір використовуваного виводу визначається дизайном плати стенду. На рис. 5.5 зображено результат цього кроку.

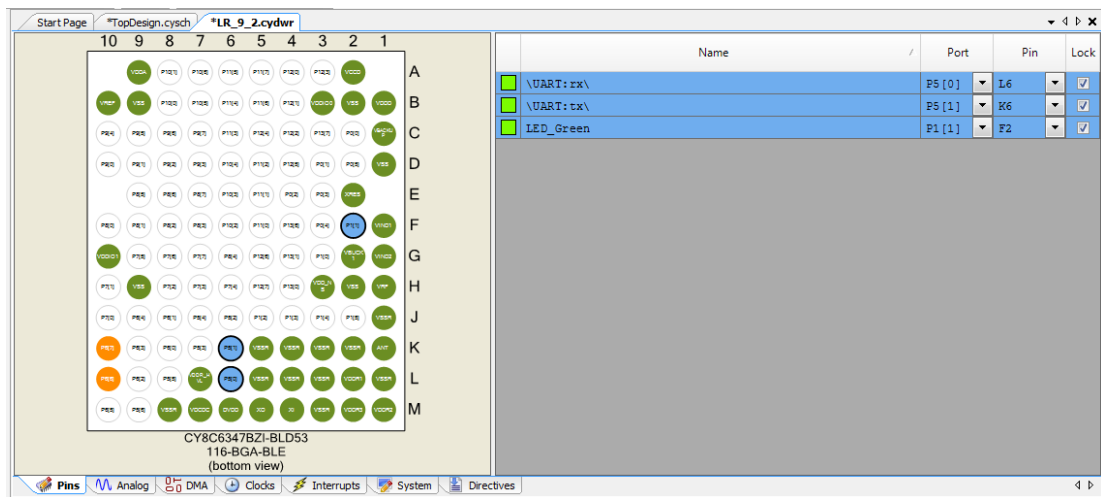


Рис. 5.5. Призначення виводів в схемі проекту

Налаштування системи переривань (рис. 5.6).

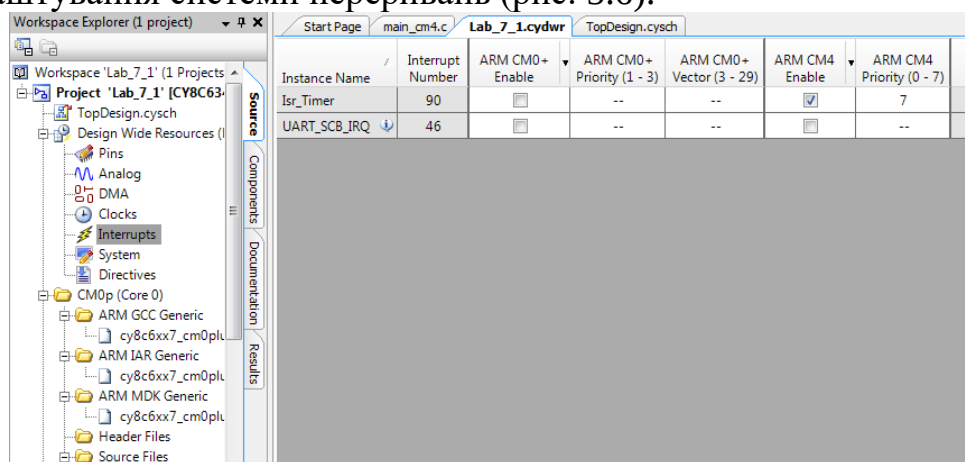


Рис. 5.6. Вікно налаштування переривань

Реалізація програмного забезпечення.

```
// LR_9_1

#include "project.h"
// Визначення макросів
#define LED_ON      (0)
#define LED_OFF     (1)

void UartInit(void);
void TimerInit(void);
void Isr_Timer(void);

// Глобальні змінні
bool LEDupdateFlag = false;

// Головна функція проекту.
// Це головна функція для процесорного ядра CM4. Запускає UART та TCPWM
// периферію. Компонента TCPWM налаштована як таймер, який генерує переривання
// кожної секунди. При кожному перериванні змінюється стан світлодіоду.
// На протязі 1 сек LED_Green світить, а на протязі наступної – не світить.

int main(void)
{
    // Запуск UART периферії
```



```

    UartInit();

    // Запуск глобальних переривань
    __enable_irq();

    // \x1b[2J\x1b[H - ANSI ESC послідовність для очищення екрану
    Cy_SCB_UART_PutString(UART_HW, "\x1b[2J\x1b[H");
    Cy_SCB_UART_PutString(UART_HW, "*** Example Project UART ***\r\n\n");
    Cy_SCB_UART_PutString(UART_HW, " --- Lab_Work_9 ---\r\n\n");
    Cy_SCB_UART_PutString(UART_HW, "Press Enter key to start blinking the
LED\r\n\n");

    // Очікування, поки користувач натисне клавішу Enter
    while(Cy_SCB_UART_Get(UART_HW) != '\r');

    // Запуск TCPWM периферії. Компонента TCPWM налаштована як таймер
    TimerInit();
    Cy_SCB_UART_PutString(UART_HW, "Observe the LED blinking on the
kit!!!\r\n");

    for(;;)
    {
        if(LEDUpdateFlag)
        {
            // Очищення прапорця
            LEDUpdateFlag = false;

            // Зміна стану світлодіода LED
            Cy_GPIO_Inv(LED_Green_0_PORT, LED_Green_0_NUM);
        }
    }
}

// Функція UartInit. Ця функція ініціалізує периферійні пристрої UART з
// налаштуваннями конфігурації UART, визначеної в структурі UART_config.
// Вона також запускає UART периферію.
void UartInit(void)
{
    /* Конфігурація UART периферії.
    Структура UART_config визначається компонентою UART_PDL на основі
    параметрів, введених в конфігурацію компоненти */
    Cy_SCB_UART_Init(UART_HW, &UART_config, &UART_context);
    /* Запуск UART периферії */
    Cy_SCB_UART_Enable(UART_HW);
}

/* Функція TimerInit. Ця функція конфігурує та запускає TCPWM периферію в
режимі таймера. Вона також конфігурує та відображає переривання TCPWM на
процесорне ядро CM4. */

void TimerInit(void)
{
    /* Конфігурація TCPWM периферії.
    Структура Counter_config визначається на основі введених параметрів
    в конфігурації компоненти */
    Cy_TCPWM_Counter_Init(Timer_HW, Timer_CNT_NUM, &Timer_config);

    /* Запуск ініціалізації лічильника */
    Cy_TCPWM_Counter_Enable(Timer_HW, Timer_CNT_NUM);

    /* Запуск увімкненого лічильника */
    Cy_TCPWM_TriggerStart(Timer_HW, Timer_CNT_MASK);

    /* Налаштування ISR для периферійного пристрою TCPWM */

```

```

Cy_SysInt_Init(&SysInt_Timer_cfg, Isr_Timer);

/* Включення переривання в NVIC */
NVIC_EnableIRQ((IRQn_Type)SysInt_Timer_cfg.intrSrc);
}

// Функція Isr_Timer. Це функція обробника переривань для переривання TCPWM.
void Isr_Timer(void)
{
    /* Очищення периферійного переривання TCPWM */
    Cy_TCPWM_ClearInterrupt(Timer_HW, Timer_CNT_NUM, CY_TCPWM_INT_ON_TC);

    /* Очищення очікуваних переривань CM4 NVIC для TCPWM */
    NVIC_ClearPendingIRQ(SysInt_Timer_cfg.intrSrc);
    LEDUpdateFlag = true;
}

```

Символи управління (*Control characters*) – елементи кодування символів, які не відображаються графічно, але які використовуються для керування пристроями, організації пересилання даних й інших цілей.

Символи управління ASCII:

\b - повернення на один крок (Переміщає позицію курсора на один символ назад та стирає попередній символ, якщо він існував.);

\t - горизонтальна табуляція (Переміщає позицію курсора до наступної позиції горизонтальної табуляції.)

\n - зміна рядка (Переміщає позицію курсора на один рядок вниз.)

\v - вертикальна табуляція (Переміщає позицію курсора до наступної позиції вертикальної табуляції.)

\r - повернення каретки (Переміщає позицію друку в крайнє ліве положення.)

\x1b – Escape (Означає, що наступні за ним символи мають якесь інше значення, відмінне від визначеного в ASCII. За звичай починає послідовності управління.)

Escape послідовності:

[l;sH - встановлює курсор в *s*-ту позицію *l*-го рядка. За замовчуванням ([H) курсор встановлюється в верхній лівий кут екрана.

[nA - переміщає курсор вгору на *n* рядків без зміни позиції всередині рядку. За замовчуванням ([nA) приймається рівне 1. Якщо курсор вже знаходиться в верхньому рядку, то послідовність ігнорується.

[nB - переміщає курсор вниз на *n* рядків без зміни позиції всередині рядку. За замовчуванням ([nB) приймається рівне 1. Якщо курсор вже знаходиться в нижньому рядку, то послідовність ігнорується.

[nC - переміщає курсор вздовж рядка на *n* позиції вправо. За замовчуванням ([nC) приймається рівне 1. Якщо курсор вже знаходиться в крайній правій позиції рядка, то послідовність ігнорується.

[nD - переміщає курсор вздовж рядка на *n* позиції вліво. За замовчуванням ([nD) приймається рівне 1. Якщо курсор вже знаходиться в крайній лівій позиції рядка, то послідовність ігнорується.

[s] - запам'ятовує поточну позицію курсора (номер рядка і колонки) для забезпечення можливості наступної його вставки в цю позицію. Більше однієї позиції запам'ятати не можна.

[u] - відновлює позицію курсора (встановлює курсор в позицію, яка запам'яталася останньою з послідовностей **[s]**).

[6n] - виводить на екран поточну позицію курсора у вигляді **[l;sR]**, де **l** – номер рядка, а **s** – номер стовпця.

[2J] - очищає екран і встановлює курсор в його лівий верхній кут.

[K] - удаляє всі символи від курсора до кінця рядка включаючи позицію курсора.

Для реалізації зв'язку між комп'ютером та PSoC 6 не достатньо лише правильно забезпечити з'єднання UART. Для візуального спостереження пакету даних, які надходять з PSoC на комп'ютер та для формування даних які надсилаються в зворотньому напрямку можна скористатися термінальною програмою «Putty».

Розглянемо налаштування, які потрібно здійснити для того, щоб реалізувати двосторонній обмін інформації між комп'ютером та PSoC 6.

Програма Putty не потребує інсталяції. Після її запуску відкриється вікно, зображене на рис. 5.7.

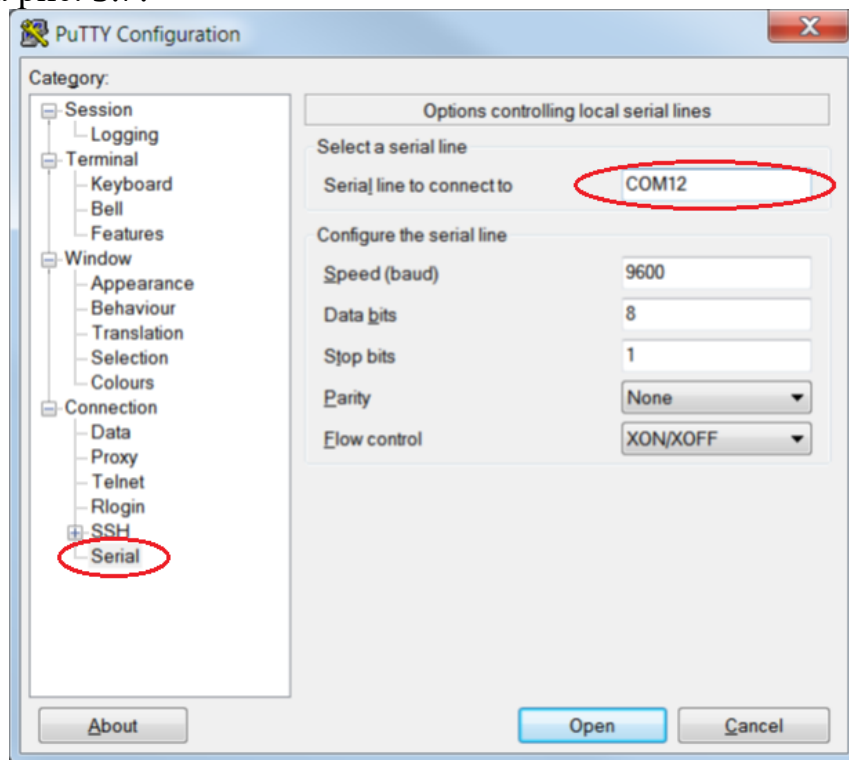


Рис. 5.7. Вигляд головного вікна програми Putty

В даному вікні в блоці “Category” вибираємо категорію “Serial” як вказано на рисунку, а в блоці “Serial line to connect to” вказуємо відповідний номер віртуального COM- порту, до якого підключений стенд PSoC 6. Якщо номер порту невідомий, то його можна взнати зайшовши в меню "Диспетчер пристроїв". Вказавши відповідне значення COM- порту, до якого приєднано

PSoC 6. В пункті “Speed (baud)” вказуємо значення швидкості передачі даних. Це значення повинно співпадати із значенням, яке вказується при налаштуванні модуля UART.

Після внесення необхідних змін переходимо на вкладку “Session” та ставимо прапоретъ в блоці “Serial”, як це вказано на рис. 5.8. Після цього у колонках “Serial line” та “Speed” автоматично запишуться вказані раніше значення COM-порта та швидкості передачі даних.

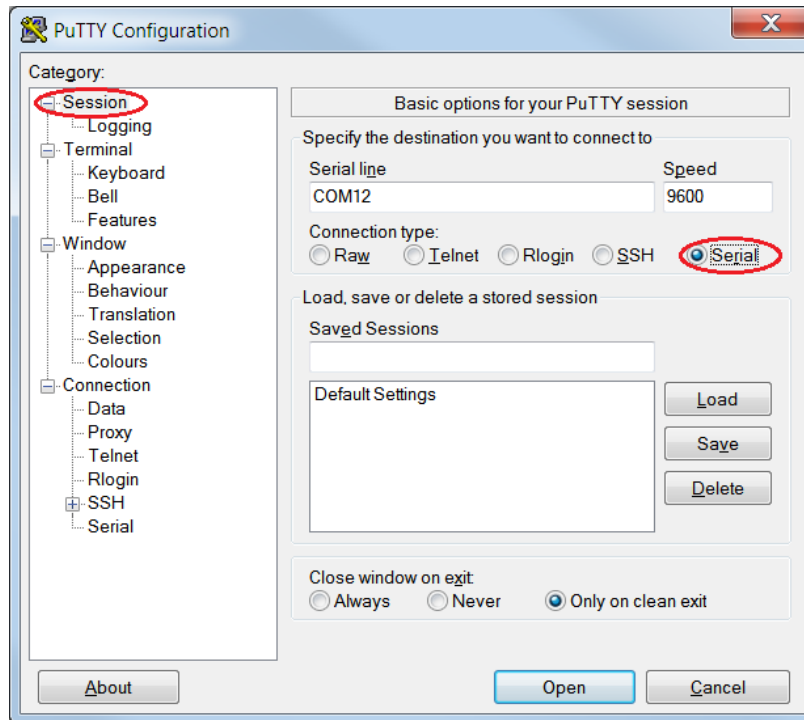


Рис. 5.8. Налаштування даних вкладки Session програми Putty

Після завершення налаштувань у категорії “Terminal” в підкатегорії “Line discipline options” налаштування встановлюються згідно рис. 5.9.

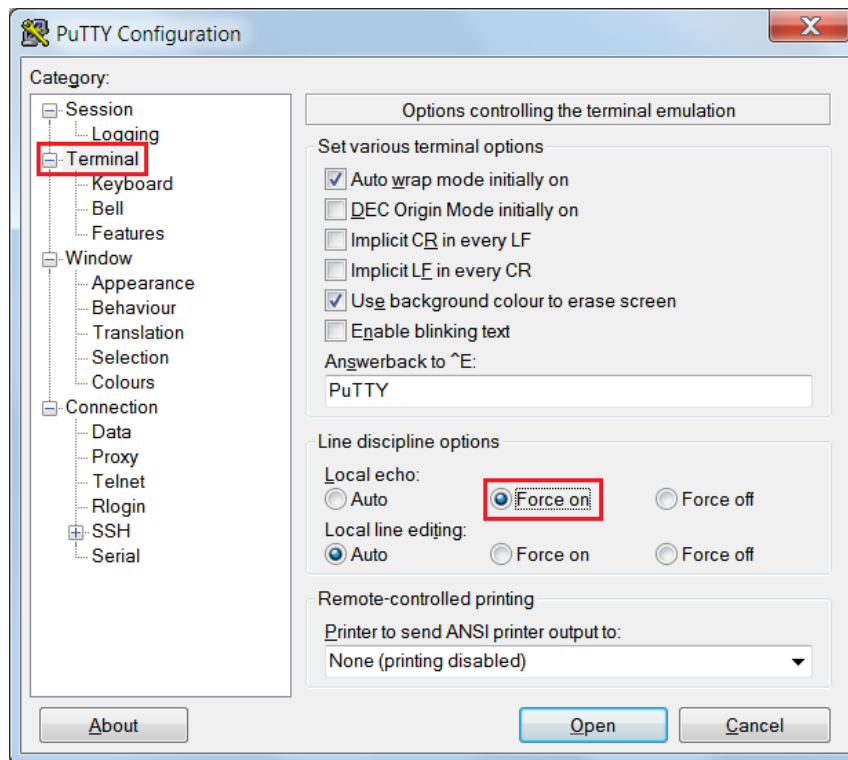


Рис. 5.9. Налаштування вкладки Terminal програми Putty

Після виконання всіх налаштувань натискаємо на кнопку “Open”, після чого появляється вікно з командною стрічкою, яке відображає дані, отримані з мікроконтролера PSoC 6, та дозволяє надсилати дані в зворотньому напрямку.

Завдання.

1. Реалізувати проект засвічування RGB світлодіодів стенду PSoC 6 BLE PIONER KIT. Алгоритм засвічування світлодіодів: при включенні живлення стенду на термінал виводиться інструкція про роботу цього проекту та дані про розробника. Жодний з RGB світлодіодів не засвічується. При натисканні на кнопку 'r' клавіатури починає блимати червоний світлодіод з частотою 0.5 Гц. При натисканні на кнопку 'b' клавіатури починає блимати синій світлодіод з частотою 1.0 Гц. При натисканні на кнопку 'g' клавіатури починає блимати зелений світлодіод з частотою 2.0 Гц. При натисканні на кнопку Enter блимання світлодіодів припиняється.

2. Реалізувати проект засвічування RGB світлодіодів стенду PSoC 6 BLE PIONER KIT з використанням переривань виводу P0.4, до якого під'єднано перемикач SW_2. Алгоритм засвічування світлодіодів: при включенні живлення стенду на термінал виводиться інструкція про роботу цього проекту та дані про розробника. Жодний з RGB світлодіодів не засвічується. При натисканні на кнопку 'r' клавіатури починає блимати червоний світлодіод з частотою 0.5 Гц. При натисканні на кнопку 'b' клавіатури починає блимати синій світлодіод з частотою 0.5 Гц. При натисканні на кнопку 'g' клавіатури починає блимати зелений світлодіод з частотою 0.5 Гц. При натисканні на кнопку Enter блимання світлодіодів припиняється. Кожний зі світлодіодів

блимає з однаковою частотою. При першому натисканні на кнопку SW_2 частота блимання змінюється на 2.0 Гц. Після другого натискання на кнопку SW_2 частота блимання стає знову 0.5 Гц і т. д. Кожна дія проекту супроводжується виводом інформації на термінал.